RECURSION & EXHAUSTIVE SEARCH (BRUTE FORCE)

ปราการ อุณจักร
ภาควิชาวิทยาการคอมพิวเตอร์, คณะวิทยาศาสตร์, มหาวิทยาลัยเชียงใหม่

Outline

- Optimization Problem
- Recursion
- Exhaustive Search
  - Search Space
  - การเขียนโปรแกรมกระทำ Exhaustive Search
  - ข้อดีข้อเสีย
- Branch-and-bound Search

Optimization Problem

- คือ การค้นหา (Search) คำตอบ (Solution) ที่ดี/เหมาะสมที่สุดในปัญหาที่ให้มา
  - สามารถวัด**คุณภาพ**ของคำตอบได้

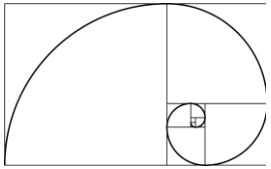
  และ/หรือ

  - คำตอบต้องผ่าน**เงื่อนไขข้อจำกัด**จึงจะยอมรับได้
- ตัวอย่าง:
  - การหาเส้นทางที่**สั้น**ที่สุด จากจุด A ไปจุด B ในกราฟ
  - การจัดตารางเรียนให้ได้**จำนวนวิชาเรียนมากที่สุด**โดยที่**ไม่มีวิชาใด ๆ ใช้ห้องเรียนเดียวกันในเวลาเดียวกัน**
  - หาจำนวนเต็ม x **ที่ทำให้สมการ $x^2 - 3x + 2 = 0$ เป็นจริง**

Recursion

- การเรียกซ้ำ (Recursion) เป็นการแจกแจงปัญหาที่มาจากปัญหาเดียวกันที่ “มีขนาดเล็กลง”
- Recursion จะถูกแทนด้วยชื่อค่าที่สนใจ และค่า Argument (ข้อมูลเข้า) ของปัญหา
- ในการทำงาน Recursion จะเรียกหาคำตอบของปัญหาย่อย และเมื่อได้ค่ากลับมาแล้ว ก็จะมีการรวบรวมค่าเหล่านั้นเพื่อให้ได้คำตอบต่อไป (บวกรวม ใช้ค่ามาก/น้อยที่สุด หรือ อย่างอื่น)
  - จะหยุดการเรียกซ้ำเมื่อพบกับ Base Case
- เราสามารถใช้ Recursion มาช่วยแก้ปัญหาได้หลายประเภท และจะเป็นส่วนหนึ่งของเทคนิคต่าง ๆ ที่จะได้เห็นในค่ายนี้

Fibonacci Numbers



- Fibonacci number ได้ค่ามาจากผลบวกของ Fibonacci number ตัวก่อนหน้าสองตัว

รูปสมการ Recursive Form ของ Fibonacci Number

$$fibo(n) = fibo(n-1) + fibo(n-2)$$

- รวบรวมผลด้วยการบวก
- จะได้ fibo(n-1) = fibo(n-2) + fibo(n-3) ไปเรื่อย ๆ
- แต่ยังต้องมีองค์ประกอบอีก 1 ส่วนคือ การเรียกซ้ำจะหยุดเมื่อใด – Base Case(s) สำหรับ Fibonacci Number นิยามให้

$$fibo(0) = 0$$
$$fibo(1) = 1$$

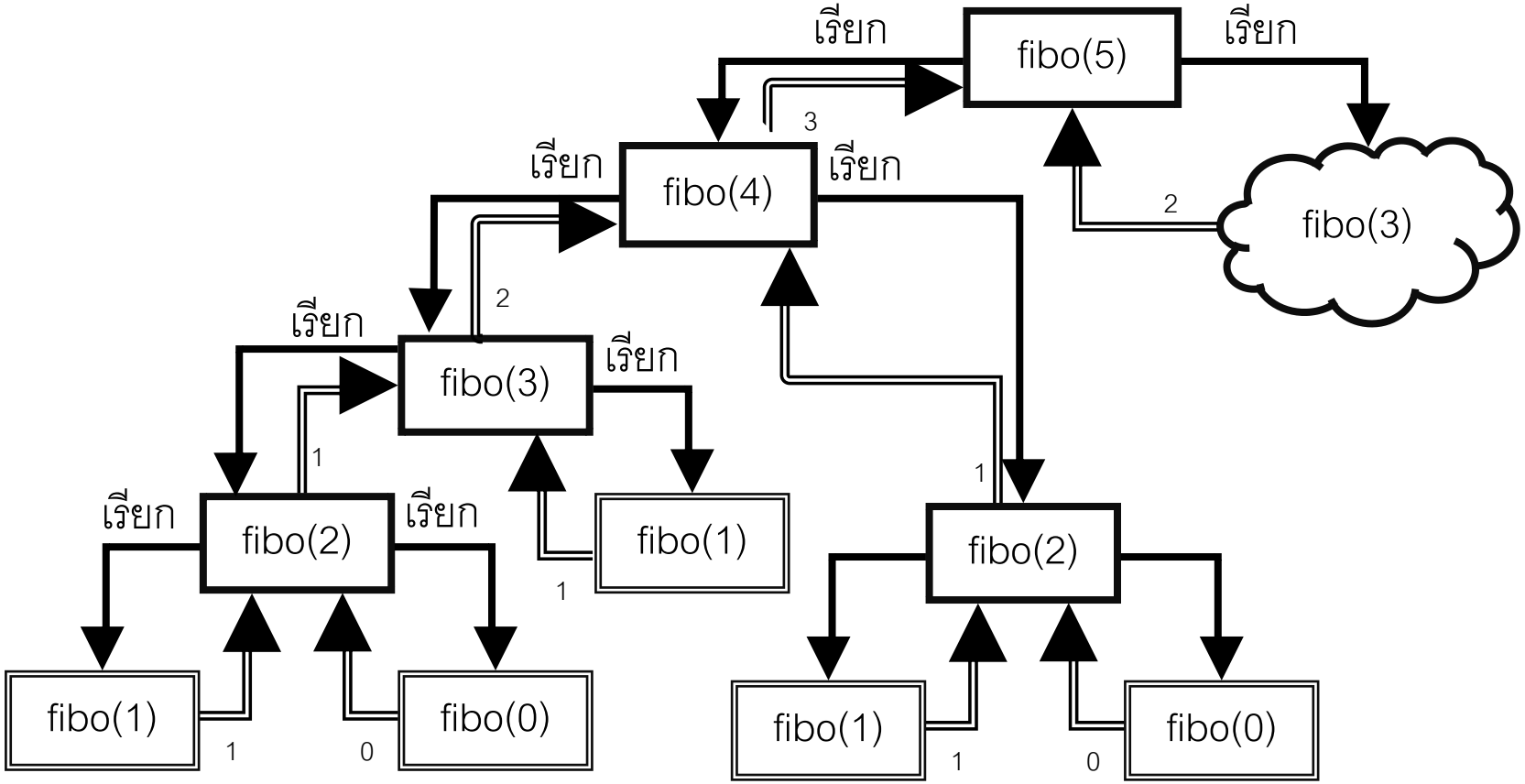
Fibonacci Numbers (ต่อ)

■ โค้ด

```
function fibo(n)
    if n = 0 then return 0
    else if n = 1 then return 1
    else return fibo(n - 1) + fibo(n - 2)
```

n	0	1	2	3	4	5	6	7
fibo(n)	0	1	1+0 = 1	1+1 = 2	2+1 = 3	3+2 = 5	5+3 = 8	8 + 5 = 13

ตัวอย่างการคำนวณ – หา fibo(5)

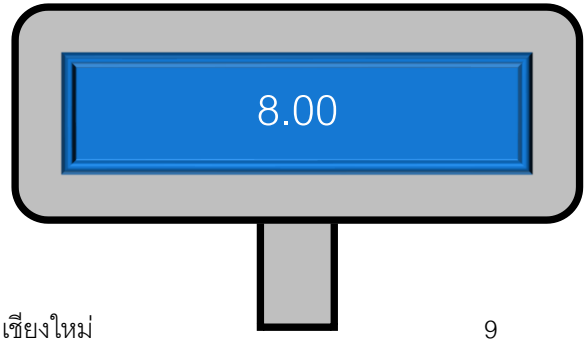


การทำงานของ Recursion

- ในการเรียกฟังก์ชัน ข้อมูลต่าง ๆ ของผู้เรียกจะถูกเก็บไว้ใน Call Stack
- ถ้ามีการเรียกและรอผลกลับ ขนาดของ Call Stack ก็จะใหญ่ขึ้นเรื่อย ๆ และอาจเกิด Error ได้
- เมื่อได้รับผลคืนมา ตัวแปรต่าง ๆ ก็จะได้รับค่ากลับจาก Call Stack ทั้งนี้ต้องระวังค่าของตัวแปรที่เราผ่านค่า Pointer ไป เช่น Array ซึ่งค่าตัวแปรจะไม่ได้เก็บใน Call Stack และการเปลี่ยนแปลงระหว่างทางจะเปลี่ยนไปเลย
  - ถ้าต้องการเปลี่ยนสถานะกลับ (Backtracking) จะต้องมีการจัดการส่วนนี้เพิ่มเติม

ตัวอย่าง #2: การทอนเงิน

- ตอนนี้จะดูเฉพาะกรณีเลขจำนวนเต็ม
- ใช้ชุดเหรียญมูลค่า c[0], c[1],…, c[k-1] เรียงจากค่าน้อยไปค่ามาก
- จงหา**<u>จำนวนเหรียญที่น้อยที่สุด</u>**ที่สามารถทอนเงิน *n* บาทได้
- ปัญหานี้ มีตัวอย่างที่ Greedy Algorithm ไม่สามารถให้ Optimal Solution ได้ – จำเป็นต้องพิจารณาตัวเลือกอื่นด้วย



การทอนเงิน (ต่อ)

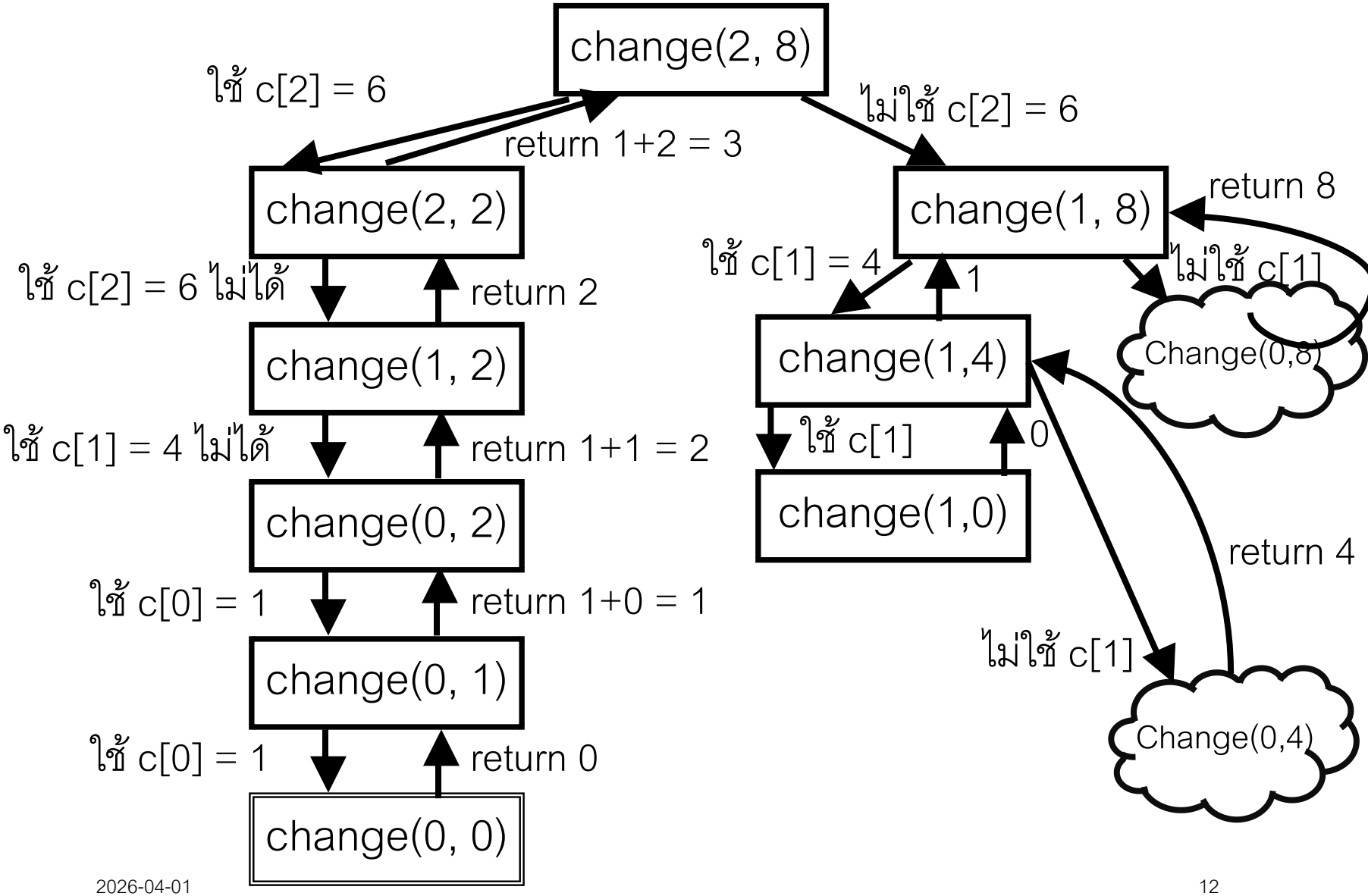
- พิจารณาเหรียญจากมูลค่ามากไปหาน้อย (ทำไมถึงทำได้) ในแต่ละขั้นตอน
  - มีตัวเลือกอะไรบ้าง
    - ใช้/ไม่ใช้เหรียญที่ index k
  - เมื่อทำตัวเลือกนั้นไปแล้ว เกิดอะไรขึ้นบ้าง
    - ใช้เหรียญที่ index k (ถ้าใช้ได้)
      - ใช้ 1 เหรียญบวกเงินทอนของ n = n – c[k]
    - ไม่ใช้เหรียญที่ index k
      - จะต้องทอนเงิน n โดยพิจารณาเหรียญที่ index k – 1
  - รวบรวมผลด้วยการเลือกค่าที่น้อยที่สุด
- Base Case
  - ทอนเงินครบแล้ว (n = 0) – Return 0 – เท่านั้น?
    - ถ้า c[0] > 1 อาจต้องพิจารณากรณีที่ n < c[0] ด้วย

การทอนเงิน (ต่อ)

- ตอนนี้ จะดูแค่ในกรณีที่ c[0] = 1 – ทอน n ที่เป็นจำนวนเต็มบวกได้เสมอ
- Recursion – ให้ change(k, n) เป็นจำนวนเหรียญน้อยที่สุดที่ใช้ c[0] … c[k] ทอนเงิน n บาท
- change(k, n) =
  - ถ้า k = 0, 1 + change(k, n – c[0])
  - ถ้า k > 0
    - และ c[k] > n (ใช้เหรียญนี้**<u>ไม่</u>**ได้), change(k – 1, n)
    - และ c[k] $\leq$ n (ใช้เหรียญนี้ได้),

      min(1+change(k, n- c[k]), change(k-1, n))

- ตัวอย่าง การทอนเงิน n = 8 บาท ด้วยเหรียญ 1, 4, และ 6 บาท (k = 0, 1, 2) – เริ่มที่ change(2, 8)



ตัวอย่าง #3 : 0/1 Knapsack

- มีเป้ (knapsack) รองรับน้ำหนักได้ W
- มีเซต S: สินค้าที่แบ่งไม่ได้อยู่ n ชิ้น
  - แต่ละชิ้น i มีน้ำหนัก w_i และมีมูลค่า v_i
- จะนำของใส่เป้อย่างไรให้มีมูลค่ารวมมากที่สุด โดยที่น้ำหนักรวมไม่เกิน W

  ทำให้ $\sum_{i=1}^{n} x_i v_i$ ได้ค่ามากที่สุด โดยที่ $\sum_{i=1}^{n} x_i w_i \leq W$
  - $x_i = 1$ ถ้าถูกเลือก, $x_i = 0$ ถ้าไม่ถูกเลือก
- 0/1 หมายความว่า ไม่สามารถแบ่งของได้ ต้องไม่เลือก (0) หรือเลือกทั้งชิ้นเท่านั้น

ตัวอย่าง #3: 0/1 Knapsack (ต่อ)

- ตัวอย่าง
- $W = 11$

i	v_i	w_i	v_i / w_i
1	1	1	1.00
2	6	2	3.00
3	18	5	3.60
4	22	6	3.67
5	28	7	4.00

0/1 Knapsack (ต่อ)

- ตัวเลือก – พิจารณาของแต่ละชิ้น i ทำอะไรได้บ้าง ทำแล้วเกิดอะไรขึ้น
  - ใช้ – ใช้ได้เมื่อ?
    - $W \geq w[i]$
    - ผล – $W = W - w[i]$ และ $V = V + v[i]$
    - พิจารณาของชิ้นต่อไป (i+1)
  - ไม่ใช้
    - W, V ไม่เปลี่ยน
    - พิจารณาของชิ้นต่อไป (i+1)

0/1 Knapsack (ต่อ)

- Recursion ให้ $Knapsack(W, i)$ แทนมูลค่าที่ดีที่สุดเมื่อกระเป๋ายังรับน้ำหนักได้ W และกำลังพิจารณาของชิ้นที่ i, i = 0…n-1
  - Base Case
    - W = 0, return 0
    - i $\geq$ n, return 0
  - Recursive Form - รวบรวมผลด้วยการใช้ค่ามากที่สุด

$$Knapsack(W, i) = \begin{cases} W < w[i], Knapsack(W, i-1) \\ W \geq w[i], \max\begin{pmatrix} v[i] + Knapsack(W - w[i], i-1) \\ , Knapsack(W, i-1) \end{pmatrix} \end{cases}$$

Exhaustive Search

- บางทีเรียก Brute Force Search หรือ Generate-and-test
- คือ การสร้างและทดสอบคำตอบที่เป็นไปได้ทั้งหมด เพื่อหาคำตอบที่ดีที่สุด/ถูกต้องตามเงื่อนไข
- ตัวอย่าง:
  - การหาค่า x ที่ทำให้สมการเป็นจริง ด้วยการลองแทนค่า x ทีละตัว
  - ค้นหาเส้นทางกลับบ้านจากโรงเรียนที่สั้นที่สุด โดยการเปรียบเทียบระยะทางของเส้นทางที่เป็นไปได้ทั้งหมด
  - การเดารหัสพาสเวิร์ดของเพื่อน โดยการลองใส่รหัสที่น่าจะถูกต้องเข้าไปทีละตัว

การทำงานของ Exhaustive Search

1. สร้างคำตอบ จาก ชุดคำตอบที่เป็นไปได้ทั้งหมด
2. ทดสอบคำตอบ
   - (ถ้าหาคำตอบที่ดีที่สุด) ถ้าคำตอบปัจจุบัน ดีกว่าคำตอบที่ดีที่สุดที่จำไว้ ให้จำคำตอบปัจจุบันเป็นคำตอบที่ดีที่สุดแทน
   - (ถ้าหาชุดคำตอบที่ผ่านเงื่อนไข) ถ้าคำตอบปัจจุบันผ่านเงื่อนไข ก็ให้เพิ่มคำตอบปัจจุบันเข้าไปในชุดคำตอบที่จำไว้ด้วย
3. ทำซ้ำข้อ 1., 2. จนกว่าจะทดสอบคำตอบที่เป็นไปได้ทั้งหมดครบ
4. คืนค่า คำตอบ/ชุดคำตอบที่ได้จำไว้

Brute Force

- หรือจะมองอีกทางได้ว่า

  "Brute Force คือการ **<u>ลองทุกอย่าง</u>**"

- ปัญหาคือ "ทุกอย่าง" ในแต่ละปัญหา คืออะไร
  - เซตของคำตอบที่เป็นไปได้ทั้งหมด จะเรียกว่า Search/Solution Space
  - สามารถใช้ Recursion มาทำการหาคำตอบด้วยวิธีนี้ (และวิธีอื่น ๆ) ได้

Search Space

- หรือ Solution Space คือ เซตของ คำตอบที่เป็นไปได้ทั้งหมด ในปัญหานั้นๆ
  - คำตอบบางตัว อาจดีกว่าตัวเลือกอื่นๆ ได้
- การทำ Exhaustive Search จะต้องมีวิธีการสร้างคำตอบ**ทุกตัว** ใน Search Space เพื่อที่จะสามารถหาคำตอบที่ดีที่สุดได้
- ดังนั้น เมื่อได้รับปัญหามา
  - จะต้องนิยาม Search Space ได้ (คำตอบจะมีลักษณะอย่างไร)
  - ควรรู้ขนาดของ Search Space ด้วย
  - จะนิยาม Search Space ให้ครอบคลุมคำตอบที่ถูกต้องทั้งหมด โดยที่มีขนาดเล็กที่สุดเท่าที่จะเป็นไปได้

ตัวอย่าง #1: Brute force รหัสพาสเวิร์ด

- ถ้าเป็นเลข PIN (4 หลัก)
  - Search Space: เลข 0000 – 9999
  - **ขนาดของ** Search Space: ตัวเลข 10,000 ตัว
- ถ้าเป็นตัวเลขฐานสองไม่เกิน 8 หลัก
  - Search Space: 0, 1, 00, 01, …. 11111111
  - **ขนาดของ** Search Space: ???
- ถ้าเป็นตัวอักษรภาษาอังกฤษกับตัวเลขไม่เกิน 4 หลัก
  - Search Space: ???
  - **ขนาดของ** Search Space: ???
- ถ้าเป็นตัวเลขหรือคำศัพท์ที่เกี่ยวข้องกับเจ้าของรหัส
  - Search Space?
  - ขนาด? 100, 1,000, 10,000?

ตัวอย่าง #2: Closest Pair

- ให้จุดบนพิกัดมา *n* จุดจงหาคู่ของจุดสองจุดที่อยู่ใกล้กันที่สุด
- Search Space:
  - คู่ของจุดพิกัดทั้งหมด
- ขนาดของ Search Space:
  - $C(n, 2) = \frac{n \times (n-1)}{2}$

จุดที่	พิกัด
1	(1, 5)
2	(4, 7)
3	(3, 2)
4	(0, 8)
5	(6, 5)

0/1 Knapsack

knapsack.cpp

```
struct ksItem{
    int v;  // value
    int w;  // weight
    double vPerW;   // value/weight ratio
};
```

```
// Exhaustive version, i is the ith (0..n-1) item being considered
// Assume objects cannot be divided
double knapsackRecursive(int W, vector<ksItem> itemArray, int i){
    double value = 0.0;

    if(W==0 || i >= itemArray.size()) {
        return 0;
    }   else {
        // 1st case: don't use ith item
        value = knapsackRecursive(W, itemArray, i+1);

        // 2nd case: use (if possible)
        if(W >= itemArray[i].w){
            double newValue = itemArray[i].v
                + knapsackRecursive(W - itemArray[i].w, itemArray, i+1);
            if(newValue > value){
                value = newValue;
            }
        }
    }
    return value;
}
```

ตัวอย่าง #3: 0/1 Knapsack

- และสามารถไล่ดูคำตอบที่เป็นไปได้ทุกคำตอบ (Solution Space) ได้โดยตรง – Enumeration
- โดยให้คำตอบเป็น Array Boolean *choice* ขนาด n
  - choice[i] = ใช้ของชิ้นที่ i หรือไม่
  - ต้องตรวจว่า น้ำหนักรวมเกินหรือไม่
  - แทนด้วยเลขฐานสองขนาด n บิตได้
    - 000…000 ไม่ใช่เลย → 111…111 ใช้ทุกชิ้น
  - ไล่ดูทีละคำตอบ ถ้า (1) น้ำหนักไม่เกิน W และ (2) ผลรวมมูลค่ามากกว่าค่าดีที่สุดที่เคยเห็น ให้เป็นค่าที่ดีที่สุดใหม่

```cpp
// Enumerate all possible answers, using itemArray.size()-bit binary to represent choice
double knapsackEnumerate(int W, vector<ksItem> itemArray){
    vector<bool> choice; // choice[0] is using itemArray[0] or not, and so on
    choice.resize(itemArray.size());

    int nLoop = exp2(itemArray.size()); // number of choices is 2^itemArray.size()
    int totalW = 0; int totalV = 0; int maxV = 0; // contain answers
    int tempC;
    // run from 0 (000...000 - using nothing) -> 000.001 -> 111...111 using - everything
    for(int c = 0; c < nLoop; c++){
        totalW = 0;
        totalV = 0;
        tempC = c;
        // convert c into choices
        for(int i = 0; i < itemArray.size(); i++){
            choice[i] = (tempC % 2 == 1); // true if using that item
            tempC = tempC / 2;

        }
        // calculate total weight, values
        for(int i = 0; i < itemArray.size(); i++){
            if(choice[i]){
                totalW += itemArray[i].w;
                totalV += itemArray[i].v;
            }
        }
        // if value is better AND the weight limit is not exceeded, update maxV
        if (totalW <= W && totalV > maxV){
            maxV = totalV;
        }
    }
    return maxV;
}
```

ตัวอย่าง #4: ทอนเงิน

- Solution/Search Space?
  - คำตอบที่เป็นไปได้ จะคิดได้ยากกว่าข้อ 0/1 Knapsack
- แต่สามารถแก้ปัญหาแบบ Recursion ได้
- พิจารณาโค้ด changeRec.c

การพัฒนาโปรแกรมสำหรับ Exhaustive Search

- จุดประสงค์หลัก: เขียนโปรแกรมที่:
  - สามารถสร้างคำตอบใน Search Space ได้ทั้งหมด
  - สามารถทดสอบคุณภาพของคำตอบทุกตัวได้
- Enumeration – ใช้ Loop
  - ค่อนข้างทำได้ยากสำหรับปัญหาทั่วไป
- ใช้ Recursion
  - Base Case เป็นกรณีที่สามารถหยุดการทำ Recursion และคืนผลได้
  - Recursive Case เป็นการเรียกซ้ำไปในปัญหาที่ “เล็ก” ลง
  - ในโค้ด(ควร)จะพิจารณา Base Cases ก่อน

ข้อดีข้อเสียของ Exhaustive Search

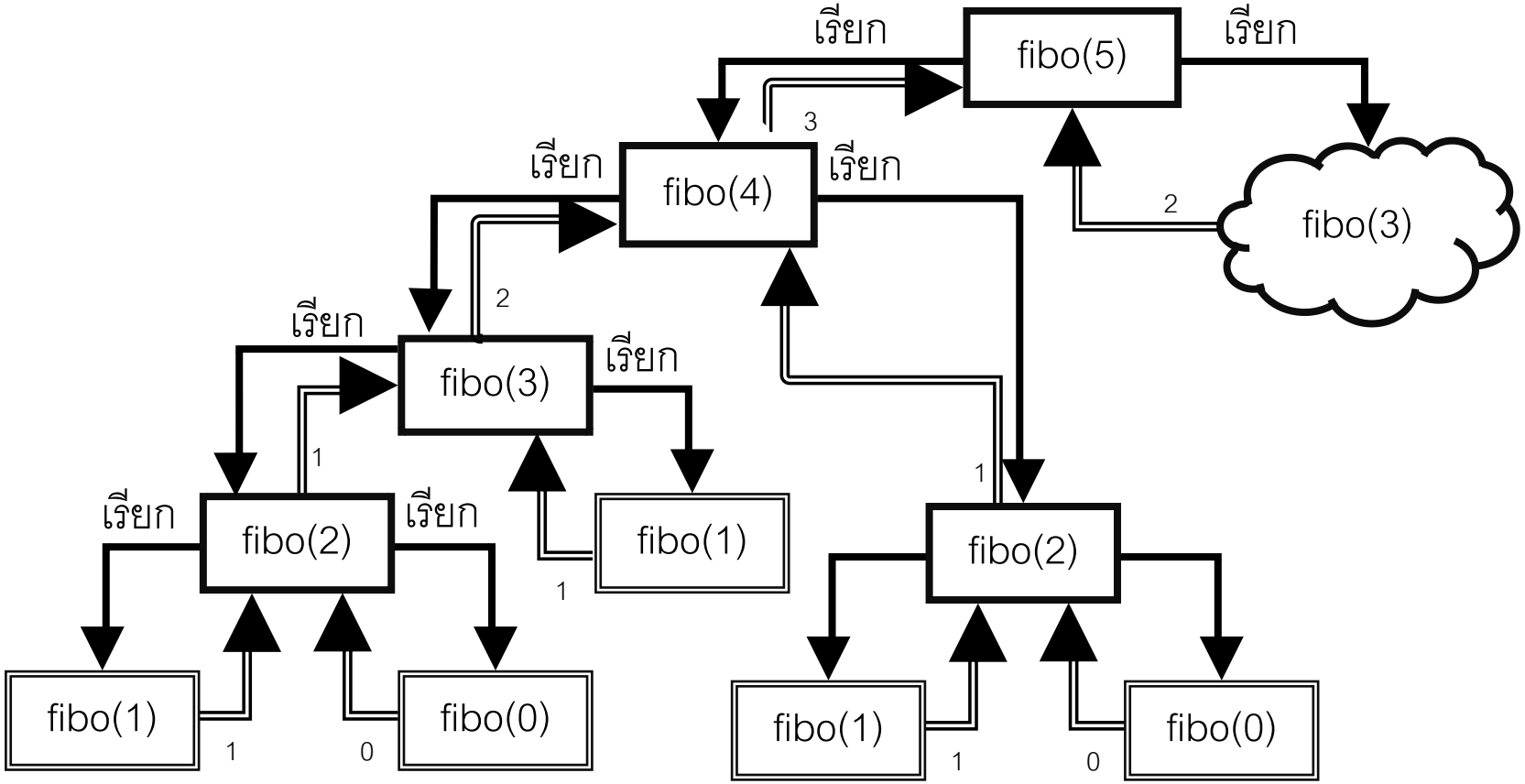
ข้อดี

- ใช้ง่าย
- แก้ปัญหาได้หลายประเภท
- สามารถแก้ปัญหาที่วิธีการอื่นไม่สามารถแก้ได้

ข้อเสีย

- ช้า มักจะไม่ได้อัลกอริทึมที่มีประสิทธิภาพสูงสุด
- อาจมีการทำการคำนวณที่ไม่จำเป็น (ซ้ำ หรือไม่มีผลกับคำตอบสุดท้าย)
- เช่น ในการแก้ปัญหา Knapsack แบบ Enumeration จะเป็นต้องพิจารณาคำตอบที่เป็นไปได้ทั้งหมดหรือไม่

พิจารณา fibo(5) มีทำซ้ำที่ใดบ้าง



Branch-and-bound Search

- Depth-first Search บน Search Tree
- ในปัญหาบางประเภท สามารถบอกได้ว่า คุณภาพของคำตอบที่ยังไม่เสร็จ (Partial Solution) จะไม่ดีไปกว่านี้ได้แล้ว
- เมื่อเจอคำตอบครั้งแรก เราจะเก็บคำตอบไว้ และอัปเดตเมื่อเจอคำตอบที่ดีกว่า
- ถ้า Partial Solution ไม่สามารถจะดีไปกว่าคำตอบที่เรามีอยู่ ก็ไม่จำเป็นที่จะต้องทำต่อไปอีก สามารถตัดการทำงาน (Prune) ไปได้บ้าง

สรุป

- Recursion
- Exhaustive Search (Brute Force)
  - ลองทุกคำตอบ เลือกคำตอบที่ให้ผลดีที่สุด
  - จะต้องรู้ search space
  - จะต้องมีวิธีสร้างทุกคำตอบใน search space
- Branch-and-bound
- <u>Brute force **ควรเป็นทางเลือกสุดท้าย**</u>

ปัญหาแบบฝึกหัด

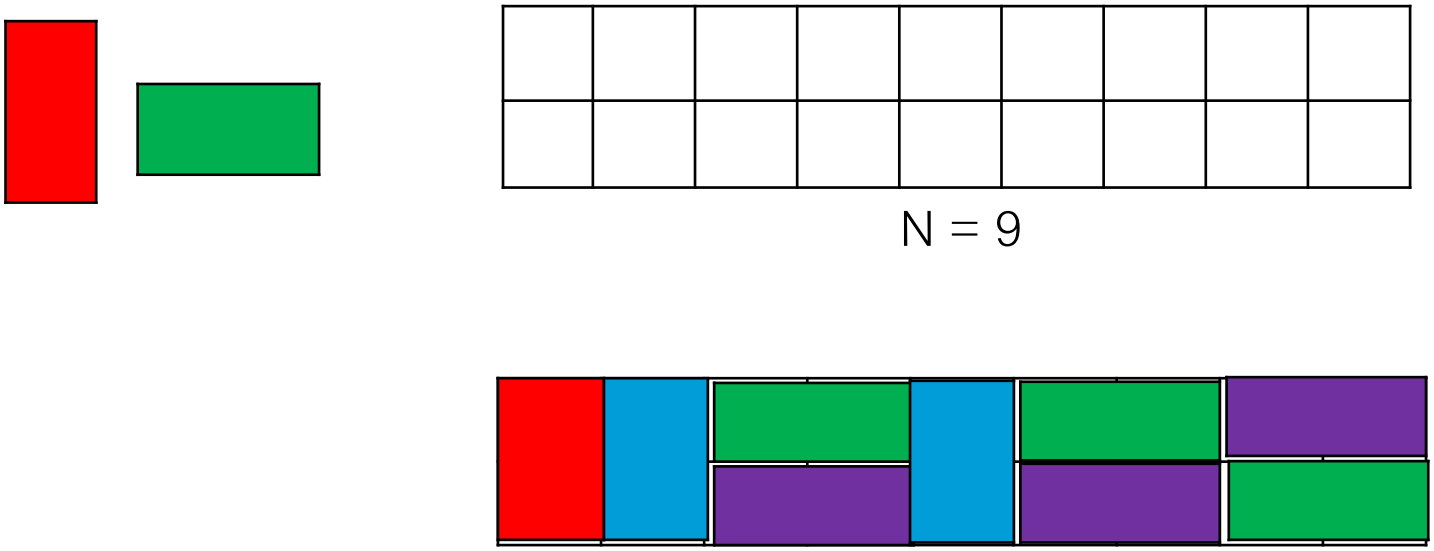
ปัญหาแบบฝึกหัด

ลองพิจารณา Recursive Form ของแต่ละปัญหา

- Tiling V0 – ปูกระเบื้องขนาด 1 × 2 ไปในพื้นที่ขนาด 2 × N
- C2P44 – จำนวนวิธีในการเขียนเลข
- C2E06 – Fruit Merchant

Tiling V0

- **มีพื้นที่ขนาด 2 ✕ N ช่อง** มีกระเบื้องขนาด 1x2 ช่อง
- จงหาจำนวนวิธีในการปูตารางโดยใช้กระเบื้อง โดยที่กระเบื้องสามารถถูกวางได้ในแนวตั้งหรือแนวนอนเท่านั้น

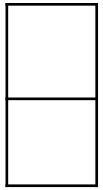


Tiling V0 (ต่อ)

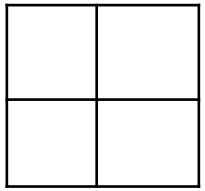
- เป็นโจทย์เรื่องการนับ → จริง ๆ ใช้ Dynamic Programming
  - แต่ควรคิด Recursive Form ให้ได้ก่อน
- จะเริ่มอย่างไร → ลองจากปัญหาเล็ก ๆ ก่อน

ลองคิดดู

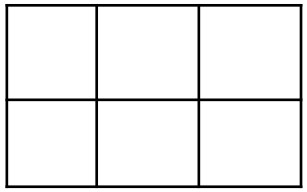
- ถ้าตารางมีขนาด 2 x 1 วางได้กี่แบบ



- ถ้าตารางมีขนาด 2 x 2 วางได้กี่แบบ



- ถ้าตารางมีขนาด 2 x 3 วางได้กี่แบบ



Tiling V0 (ต่อ)

- พิจารณาการวางตัวสุดท้าย วางได้อย่างไรบ้าง
- ถ้าตัวสุดท้ายวางแนวตั้ง

  - ไปถาม จำนวนการปูจากใครต่อ
- ถ้าตัวสุดท้ายวางแนวนอน

  - ไปถาม จำนวนการปูจากใครต่อ สองช่องข้างล่างต้องปูอย่างไร

จำนวนวิธีในการเขียนเลข (C2P44)

- มีเซตตัวเลข {1, 3, 4}
- ต้องการจำนวนวิธีที่ต่างกันในการเขียนค่า n โดยการใช้การรวมกันของตัวเลข 1, 3, 4
- ตัวอย่าง n = 5 เขียนได้ 6 วิธี

1. 1 + 1 + 1 + 1 + 1
2. 1 + 1 + 3
3. 1 + 3 + 1
4. 3 + 1 + 1
5. 1 + 4
6. 4 + 1

- มีความเหมือนหรือ**<u>ต่าง</u>**จากปัญหาทอนเงินอย่างไร
- จะเขียน Recursive Form ได้อย่างไร

Problem – Fruit Merchant (C2E06)

- เก็บลูกท้อวิเศษ ใส่ตะกร้า 2 ใบที่รับน้ำหนัก W_1 และ W_2 ภายใต้เวลา T ให้ได้ผลรวมมูลค่า v_i ,มากที่สุด
  - เก็บลูกท้อได้ทีละลูก ใช้เวลา t_i
  - ลูกท้อต้องใส่ตะกร้าใดตะกร้าหนึ่งโดยที่ลูกท้อในตะกร้าต้องมีน้ำหนัก w_i รวมกันไม่เกินที่ตะกร้านั้นจะรับน้ำหนักได้

i	v_i	t_i	w_i
1	2	3	2
2	4	4	5
3	3	5	3

Problem – Fruit Merchant (C2E06)

- ทำอะไรต่อผลไม้ลูกหนึ่งได้บ้าง
- มีผลอย่างไร
- Recursive Form จะต้องมี Argument อะไรบ้าง